

# Σχεδίαση και Υλοποίηση Μη Σχεσιακής ΒΔ στο Σύστημα MongoDB

Παναγιώτης Αθανασόπουλος  
Πανεπιστήμιο Πειραιώς, Τμήμα  
Ψηφιακών Συστημάτων  
Πειραιάς, Ελλάδα  
apanagiotis170@gmail.com

Μαρίνα Νούχου  
Πανεπιστήμιο Πειραιώς, Τμήμα  
Ψηφιακών Συστημάτων  
Πειραιάς, Ελλάδα  
marinanuhu4598@gmail.com

Αναστασία Αχιλλέως  
Πανεπιστήμιο Πειραιώς, Τμήμα  
Ψηφιακών Συστημάτων  
Πειραιάς, Ελλάδα  
anactaciaachilleos@gmail.com

## Abstract

Παρουσιάζουμε σχεδίαση και υλοποίηση μη-σχεσιακής βάσης στο MongoDB για μηνύματα AIS στον Σαρωνικό (Μάιος 2017–Δεκέμβριος 2019). Μοντελοποιήσαμε τα δεδομένα σε πέντε κύριες συλλογές (vessels, ais\_dynamic, ais\_synopses, ports/harbours, regions) με στοχευμένα ευρετήρια (συμπεριλαμβανομένου 2dsphere) ώστε να υποστηρίζονται: (α) ερωτήματα «σχεσιακού τύπου», (β) χωρικά (εντός κύκλου, *K-κοντινότερα*) και (γ) χωρο-χρονικά (εγγύτητα *X* σε διάστημα *T*). Αξιολογούμε την επίδραση των ευρετηρίων και επιλογών κατάτμησης (sharding) στον χρόνο απόκρισης και δείχνουμε σημαντικές βελτιώσεις latency σε spatial/temporal σενάρια. Το dataset διατίθεται από το Zenodo [4].

## 1 Εισαγωγή

Η ανάλυση Μεγάλων Δεδομένων (Big Data) αποτελεί μια πρόκληση που έχουν να αντιμετωπίσουν ολοένα και περισσότερες επιστήμες. Με τον όρο μεγάλα δεδομένα χαρακτηρίζονται τα δεδομένα που είναι δύσκολο να διαχειριστούν με παραδοσιακές μεθόδους ανάλυσης και περιέχουν τις εξής ιδιότητες: μεγάλο όγκο (volume), μεγάλη ποικιλία (variety) και υψηλή ταχύτητα με την οποία παράγονται (velocity). Ένα τέτοιο πρόβλημα το οποίο θα διερευνηθεί στην παρούσα εργασία είναι η διαχείριση μεγάλων δεδομένων της ναυσιπλοΐας. Είναι γεγονός ότι, η ναυτιλία κατέχει σπουδαίο ρόλο στην παγκόσμια οικονομία αφού πέρα του τουρισμού είναι υπεύθυνη περίπου για το 80% του παγκόσμιου εμπορίου. Επιπλέον, η αποτελεσματική διαχείριση της ναυσιπλοΐας είναι ένα πολύ σημαντικό κεφάλαιο αφού δεν είναι μια απομονωμένη δραστηριότητα αλλά αποτελεί μέρος ενός συστήματος που επηρεάζεται από διάφορους παράγοντες και πρέπει πάντα να γίνεται αποτελεσματικά και με ασφάλεια. [1] [2]

Μεταξύ των διαφόρων μέσων επιτήρησης που χρησιμοποιούνται για την ταυτοποίηση και τον εντοπισμό πλοίων στη θάλασσα σε πραγματικό χρόνο, το Σύστημα Αυτόματης Ταυτοποίησης (AIS) έχει καταστεί υποχρεωτικό πρότυπο για τους περισσότερους τύπους πλοίων (φορτηγά, επιβατικά, αλιευτικά, κ.λπ.). Το σύστημα AIS βασίζεται στο Παγκόσμιο Σύστημα Εντοπισμού Θέσης- GPS σε συνδυασμό με αισθητήρες πάνω στο πλοίο (όπως ταχύμετρο, πιξίδα, κ.λπ.) και το χρησιμοποιούν σήμερα πάνω από μισό εκατομμύριο πλοία παγκοσμίως για να μεταδίδουν την θέση τους.[3]

Στην παρούσα εργασία, το σύνολο δεδομένων που εξετάζεται περιλαμβάνει μηνύματα AIS (Automatic Identification System), που καταγράφηκαν στην ευρύτερη περιοχή του Σαρωνικού Κόλπου, στην Ελλάδα, για περίπου 2,5 χρόνια (Μάιος 2017 – Δεκέμβριος 2019).[4] Τα δεδομένα συλλέχθηκαν μέσω σταθμού

AIS που βρίσκεται στο Πανεπιστήμιο Πειραιά, σε απόσταση 1 χλμ. από το λιμάνι του Πειραιά, και αποτελούν ένα πρότυπο σύνολο δεδομένων για ανάλυση θαλάσσιων δραστηριοτήτων. Το σύνολο δεδομένων περιλαμβάνει τρεις βασικές κατηγορίες πληροφοριών:

- (1) Δεδομένα πλοήγησης, όπως θέσεις, ταχύτητες και πορείες πλοίων.
- (2) Γεωγραφικά δεδομένα, όπως η θέση λιμανιών και η γεωγραφική κατανομή της περιοχής.
- (3) Μετεωρολογικά δεδομένα, που συνδέονται με τις καιρικές συνθήκες κατά τη διάρκεια της καταγραφής.

Επιπλέον, τα δεδομένα περιλαμβάνουν σημαντικά μετα-δεδομένα, όπως περιλήψεις τροχιών, οι οποίες καταγράφουν κρίσιμα σημεία κατά τη διάρκεια των διαδρομών των πλοίων, όπως αλλαγές στην ταχύτητα ή στην πορεία. Αυτές οι περιλήψεις είναι ιδιαίτερα χρήσιμες για την εξαγωγή συμπερασμάτων και τη λήψη αποφάσεων.

Το σύνολο δεδομένων περιλαμβάνει πάνω από 244 εκατομμύρια εγγραφές AIS, με έναν μέσο όρο άνω των 10.000 εγγραφών ανά ώρα. Αυτό καθιστά το συγκεκριμένο σύνολο δεδομένων, ίσως το μεγαλύτερο και πιο πυκνό ανοιχτό σύνολο δεδομένων AIS για αυτή τη χρονική διάρκεια. Επομένως, αποτελεί μια ιδανική βάση για την επεξεργασία και ανάλυση μεγάλων δεδομένων κινητικότητας, και αναδεικνύεται ως ένα εξαιρετικό εργαλείο για ανάλυση σε μεγάλη κλίμακα.[3]

## 1.2 Ορισμός Προβλήματος

Επιγραμματικά, το πρόβλημα που εξετάζεται στην παρούσα εργασία σχετίζεται με τη διαχείριση, επεξεργασία και ανάλυση μεγάλου όγκου δεδομένων που προέρχονται από το Σύστημα Αυτόματης Ταυτοποίησης (AIS) για τη ναυτιλιακή δραστηριότητα στην περιοχή του Σαρωνικού Κόλπου. Το σύνολο δεδομένων είναι διαθέσιμο στην ιστοσελίδα <https://zenodo.org/records/6323416>. [4] Τα δεδομένα περιλαμβάνουν πληροφορίες όπως θέσεις πλοίων, συνόψεις των τροχιών τους, στατική πληροφορία πλοίων και τύπους πλοίων, γεωγραφικά δεδομένα (νησιά, λιμάνια) και μετεωρολογικά δεδομένα. Η πρόκληση που αντιμετωπίζεται είναι η διαχείριση ενός τόσο περίπλοκου συνόλου δεδομένων που περιέχει πολύπλοκες αναπαραστάσεις με χρήση μη σχεσιακής προσέγγισης και συγκεκριμένα μέσω της MongoDB. Για τον σκοπό αυτό απαιτείται κατάλληλη μοντελοποίηση των δεδομένων, αποθήκευση, ευρετηρίαση και κατάτμηση των δεδομένων σε κόμβους ώστε τα εκάστοτε ερωτήματα να υποστηρίζονται με αποδοτικό τρόπο. Μετά την μοντελοποίηση και μεταφόρτωση των δεδομένων στη MongoDB, τα ερωτήματα που καλούνται να εξεταστούν είναι τα ακόλουθα:

- (1) Υποστήριξη ερωτημάτων σχεσιακού τύπου, όπως η εύρεση πλοίων με σημαία Ελλάδας και όνομα που περιέχει κάποιο αλφαριθμητικό.
- (2) Υποστήριξη χωρικών ερωτημάτων, όπως η εύρεση των πλοίων εντός κύκλου με δοθέν κέντρο και ακτίνα, ή η εύρεση των Κ κοντινότερων πλοίων σε δοθέν σημείο.
- (3) Υποστήριξη χωρο-χρονικών ερωτημάτων, όπως η εύρεση των πλοίων που πλησίασαν μεταξύ τους σε απόσταση X εντός ενός χρονικού διαστήματος T.

## 2 Δεδομένα & Προεπεξεργασία

Το σχήμα της zenodoDB περιλαμβάνει:

- **ais\_dynamic**: δυναμικές θέσεις/μετρήσεις (Point GeoJSON, timestamp, speed, course, heading, vessel\_id).
- **ais\_synopses**: συνοπτικά “γεγονότα”/σημεία τροχιάς ανά πλοίο (t, lon/lat ή GeoJSON).
- **vessels**: στατικά στοιχεία πλοίων (vessel\_id, name, country, MMSI/IMO, κ.ά.).
- **ship\_types**: λεξικό τύπων πλοίου (κωδικός/ομάδα/ετικέτα).
- **Γεωχωρικές συλλογές**: ports, harbours (συνήθως Point), piraeanus\_port (Polygon της λιμενικής ζώνης), regions, islands, areas, spatial\_coverage, territorial\_waters (Polygon/MultiPolygon).

**Προεπεξεργασία.** (i) ενοποίηση συντεταγμένων σε GeoJSON [lon,lat], (ii) ISODate σε timestamp, (iii) (προαιρετικό) αντιγραφή περιγραφικού ship\_type μέσα στα vessels για γρήγορα φίλτρα χωρίς \$lookup.

**Table 1: Συνοπτικά χαρακτηριστικά συλλογών (μετά την προεπεξεργασία)**

| Collection         | #Docs     | Μέγεθος | Σημείωση                                               |
|--------------------|-----------|---------|--------------------------------------------------------|
| vessels            | «6230»    | «GB»    | Στατικά στοιχεία πλοίων, ship_type ενσωματωμένο        |
| ais_dynamic        | «4305035» | «GB»    | Θέσεις AIS (GeoJSON Point), time-series                |
| ais_synopses       | «629523»  | «GB»    | Συνοπτικά γεγονότα/σημεία τροχιάς                      |
| ship_types         | «6228»    | «MB»    | Λεξικό τύπων πλοίων (κωδ., ομάδα, ετικέτα)             |
| noaa_weather       | «11160»   | «MB»    | Χρονικά/Μετεωρολογικά στοιχεία (GeoJSON Point/Polygon) |
| harbours           | «21»      | «MB»    | Πρόσδεση (GeoJSON Point)                               |
| piraeus_port       | «1»       | «MB»    | Ζώνη λιμένα Πειραιά (GeoJSON Polygon)                  |
| regions            | «1»       | «MB/GB» | Περιοχές ενδιαφέροντος (GeoJSON (Multi)Polygon)        |
| islands            | «136»     | «MB»    | Νησιά (GeoJSON Polygon)                                |
| spatial_coverage   | «76»      | «MB»    | Κάλυψη δέκτη (Polygon/MultiPolygon)                    |
| territorial_waters | «1»       | «MB»    | Χωρικά ύδατα (Polygon/MultiPolygon)                    |

## 3 Σχεδίαση Σχήματος (Μοντελοποίηση)

Η σχεδίαση των συλλογών βασίζεται σε αρχές ευελιξίας, επεκτασιμότητας και αποδοτικής αποθήκευσης χωροχρονικών δεδομένων. Στο πλαίσιο αυτό, εφαρμόστηκαν οι ακόλουθες βασικές σχεδιαστικές επιλογές

- **Ενσωμάτωση (embedding)**: Πεδία όπως το shiptype στη vessels και το vessel στη ais\_dynamic ενσωματώνονται στα έγγραφα, επιτρέποντας γρήγορα φίλτρα χωρίς την ανάγκη για \$lookup. Το vessel\_id παραμένει το authoritative κλειδί.
- **Κανονικοποίηση (normalization)**. Για επαναχρησιμοποιούμενες ή μεταβαλλόμενες τιμές, όπως οι τύποι πλοίων (ship\_types), χρησιμοποιούνται ανεξάρτητες συλλογές για καλύτερη διαχείριση και συνέπεια.
- **Γεωχωρική υποστήριξη**: Όλα τα γεωμετρικά πεδία ακολουθούν GeoJSON πρότυπο. Οι συλλογές με συντεταγμένες σε WGS84 (π.χ. ais\_dynamic, harbours, islands) υποστηρίζουν 2dsphere ευρετήρια. Σε περιπτώσεις όπως η regions, που χρησιμοποιεί προβολικές συντεταγμένες, προτείνεται η αποθήκευση γεωμετρίας WGS84 σε ξεχωριστό πεδίο (π.χ. geometry\_wgs84) για χωρικά ερωτήματα.
- **Μοντελοποίηση χρονικών δεδομένων**: Τα timestamp πεδία χρησιμοποιούν ISODate για πλήρη συμβατότητα με το MongoDB, ενώ σε ειδικές περιπτώσεις (π.χ. ais\_synopses) χρησιμοποιείται epoch (σε milliseconds) για συμπαγή αναπαράσταση και ταχύτερους υπολογισμούς.
- **Χρήση FeatureCollections**: Οι συλλογές piraeanus\_port, regions, και spatial\_coverage υιοθετούν μορφή GeoJSON FeatureCollection για ενσωμάτωση σε συστήματα GIS και συμβατότητα με APIs όπως το Mapbox ή το Leaflet.
- **Ερωτήματα υψηλής απόδοσης**: Η σχεδίαση ευνοεί τη χρήση χωροχρονικών ερωτημάτων (\$geoWithin, \$near, range queries σε timestamp) με περιορισμένες εξαρτήσεις μεταξύ συλλογών, ελαχιστοποιώντας την ανάγκη για \$lookup.

Η παραπάνω προσέγγιση επιτρέπει την ευέλικτη και επεκτάσιμη διαχείριση δεδομένων που σχετίζονται με την θαλάσσια κυκλοφορία, διατηρώντας υψηλή απόδοση και ευκολία εμπλουτισμού. Στις επόμενες υποενότητες παρουσιάζονται αναλυτικά οι βασικές συλλογές της βάσης και οι αντίστοιχες σχεδιαστικές επιλογές.

### 3.1 Συλλογή vessels

Η συλλογή vessels αποθηκεύει τα στατικά χαρακτηριστικά των πλοίων, όπως τη σημαία του πλοίου, το όνομα και τους κωδικούς αναγνώρισης. Για λόγους αποδοτικότητας, το πεδίο shiptype ενσωματώθηκε ως υπο-έγγραφο, ώστε να αποφεύγονται επιπλέον λειτουργίες lookup κατά την εκτέλεση ερωτημάτων ανά τύπο πλοίου.

**Listing 1: Παράδειγμα εγγράφου στη συλλογή vessels (από το cluster)**

```

1 {
2   "vessel_id": "2c46b2c29086c03cb774fa3fbd09f4c22a793eb7e7403
3     21b5f17518a343dfc1e",
4   "country": "Albania",
5   "shiptype": {
6     "Type Code": 52,
7     "Description": "Tug"
8   }
9 }

```

### 3.2 Συλλογή ais\_dynamic

Η συλλογή ais\_dynamic περιέχει τα δυναμικά στίγματα που προέρχονται από το σύστημα AIS, δηλαδή τη γεωγραφική θέση του πλοίου (σε μορφή GeoJSON Point), την πορεία, την ταχύτητα, το heading και τη χρονική σήμανση (timestamp). Για λόγους ευελιξίας και αποδοτικότητας, κάθε έγγραφο περιλαμβάνει επίσης ένα προαιρετικό, ενσωματωμένο στιγμιότυπο (embedded snapshot) με βασικά στοιχεία του πλοίου, ώστε να είναι δυνατά γρήγορα φίλτρα χωρίς χρήση lookup. Παρ' όλα αυτά, το πεδίο vessel\_id παραμένει το κύριο κλειδί αναφοράς και χρησιμοποιείται για τη συσχέτιση με τα υπόλοιπα δεδομένα.

**Listing 2: Παράδειγμα εγγράφου στη συλλογή ais\_dynamic (MongoDB Extended JSON)**

```

1 {
2   "_id": { "$oid": "676fecab1ce3cbb7e4ac14e9" },
3   "course": 359.6,
4   "geometry": {
5     "type": "Point",
6     "coordinates": [ 23.6350566666667, 37.943545 ]
7   },
8   "heading": 219,
9   "speed": 0,
10  "timestamp": { "$date": "2017-05-09T14:05:26.000Z" },
11  "vessel": {
12    "vessel_id": "0fe51bee65c87f2f06f33d5b42fd47dfaf7d99c7233
13      6e2ec5543f709f90a2d9b",
14    "country": "Greece",
15    "shipType": {}
16  },
17  "vessel_id": "0fe51bee65c87f2f06f33d5b42fd47dfaf7d99c72336e
18    2ec5543f709f90a2d9b"
19 }

```

### 3.3 Συλλογή ais\_synopses

Η συλλογή ais\_synopses αποθηκεύει συνοπτικά γεγονότα που εξάγονται από την ανάλυση των τροχιών των πλοίων, ώστε να παρέχεται μια πιο συμπυκνωμένη και στοχευμένη εικόνα σε σχέση με τα ακατέργαστα στίγματα της συλλογής ais\_dynamic. Τα γεγονότα αυτά περιλαμβάνουν, ενδεικτικά, μεγάλα χρονικά κενά λήψης (gaps), αφηνίδες αλλαγές ταχύτητας ή πορείας, στάσεις, ή προσεγγίσεις σε περιοχές ενδιαφέροντος.

**Listing 3: Παράδειγμα εγγράφου στη συλλογή ais\_synopses (MongoDB Extended JSON από το cluster)**

```

1 {
2   "_id": { "$oid": "6782303890f33466a5c174ad" },
3   "t": { "$numberLong": "1501534800000" },

```

```

4   "vessel_id": "9e854ccb6855e244bd694010fcb8300633e1b1b631723
5     a50f04765521c8f3a10",
6   "lon": 23.6829416667,
7   "lat": 37.9291066667,
8   "annotations": "[ 'GAP_END' ]",
9   "transport_trail": "[ { 'topic': 'datacsv_saronikos_3', '
    timestamp': 1501534800 } ]"

```

**Σχόλιο:** Κρατά «γεγονότα»/σημεία τροχιάς ανά πλοίο. Το πεδίο χρόνου t είναι \$numberLong (ms από epoch). Τα annotations και transport\_trail είναι *συμβολοσειρές* που περιέχουν λίστες.

### 3.4 Συλλογή ship\_types

Η συλλογή ship\_types λειτουργεί ως λεξικό αναφοράς για τους τύπους πλοίων. Κάθε έγγραφο περιλαμβάνει έναν μοναδικό κωδικό και την αντίστοιχη περιγραφή του τύπου (π.χ. cargo, tanker, passenger). Η κανονικοποίηση αυτής της πληροφορίας σε ξεχωριστή συλλογή εξασφαλίζει συνέπεια και διευκολύνει την επαναχρησιμοποίησή της από άλλες συλλογές, όπως την vessels.

**Listing 4: Παράδειγμα εγγράφου στη συλλογή ship\_types (MongoDB Extended JSON)**

```

1 {
2   "_id": { "$oid": "67687ec46762f23d2f794183" },
3   "Type Code": 21,
4   "Description": "Wing in ground (WIG), Hazardous category A"
5 }

```

### 3.5 Συλλογή harbours

Η συλλογή harbours περιέχει γεωχωρικά δεδομένα για λιμάνια και μαρίνες. Κάθε έγγραφο αποθηκεύεται σε μορφή GeoJSON Point, που αντιστοιχεί στη γεωγραφική θέση του λιμένα. Στα συνοδευτικά πεδία περιλαμβάνονται βασικά περιγραφικά στοιχεία, όπως το όνομα του λιμένα και, όπου είναι διαθέσιμο, κωδικοί αναγνώρισης (π.χ. LOCODE).

Η συλλογή εξυπηρετεί ερωτήματα που αφορούν την είσοδο ή την έξοδο πλοίων από συγκεκριμένα λιμάνια, καθώς και την ανάλυση κίνησης σε περιοχές με έντονη δραστηριότητα. Χρησιμοποιώντας ευρετήρια 2dsphere, μπορούν να εκτελεστούν αποδοτικά χωρικά ερωτήματα, όπως η ανίχνευση πλοίων σε μικρή απόσταση από λιμάνι.

**Listing 5: Παράδειγμα εγγράφου στη συλλογή harbours (από το cluster)**

```

1 {
2   "_id": { "$oid": "67a12ad54c8acf03ab957eaa" },
3   "geometry": {
4     "type": "Point",
5     "coordinates": [23.64899, 37.93486]
6   },
7   "properties": {
8     "Lat": 37.93486,
9     "Lon": 23.64899,
10    "Port Name": "ZEA",
11    "type": "Marine",
12    "locode": "GRMAZ"
13  }
14 }

```

### 3.6 Συλλογή piraеus\_port

Η συλλογή piraеus port αποθηκεύει τα γεωμετρικά δεδομένα που οριοθετούν τον λιμένα του Πειραιά. Τα έγγραφα είναι σε μορφή GeoJSON FeatureCollection με γεωμετρία τύπου Polygon ή MultiPolygon, ώστε να αποδίδεται με ακρίβεια η χωρική έκταση της λιμενικής ζώνης. Η συλλογή αυτή επιτρέπει τη διατύπωση χωρικών ερωτημάτων που σχετίζονται ειδικά με τον Πειραιά, όπως ο εντοπισμός πλοίων που εισέρχονται, εξέρχονται ή παραμένουν εντός των ορίων του λιμένα.

**Listing 6: Παράδειγμα εγγράφου στη συλλογή piraеus\_port (FeatureCollection με MultiPolygon)**

```
1 {
2   "_id": { "$oid": "678234a590f33466a5cb0fc5" },
3   "type": "FeatureCollection",
4   "features": [
5     {
6       "type": "Feature",
7       "geometry": {
8         "type": "MultiPolygon",
9         "coordinates": [
10          [
11            [
12              [23.6211246,37.9367436],
13              [23.6211683,37.936639],
14              [23.6211406,37.9366305],
15              [23.6213119,37.9362654],
16              [23.6215472,37.9363241],
17              [23.6222302,37.9345956],
18              [23.6225582,37.9346891],
19              [23.6234921,37.93445],
20              [23.62467,37.9372383],
21              [23.624917,37.937821],
22              [23.6257951,37.9381988],
23              [23.6258464,37.9381235],
24              [23.6249982,37.937755],
25              [23.6247707,37.9372197],
26              [23.6248508,37.9370932],
27              [23.625902,37.9367617],
28              [23.6267146,37.9369933],
29              [23.6267836,37.9371174],
30              ...
31            ]
32          ],
33          [
34            [
35              [23.6214461866084,37.940555777528836],
36              [23.6214472,37.9405931],
37              [23.62144706749831,37.94059313526924],
38              [23.6214461866084,37.940555777528836]
39            ]
40          ],
41          [
42            [
43              [23.621410302057377,37.93923417577168],
44              [23.621411346493115,37.93927264157564],
45              [23.6214103,37.9392342],
46              [23.621410302057377,37.93923417577168]
```

```
47            ]
48          ],
49          [
50            [
51              [23.621254,37.936158],
52              [23.6209926,37.9367085],
53              [23.621000531746205,37.93672686825437],
54              [23.6209925,37.9367085],
55              [23.621254,37.936158]
56            ]
57          ]
58        ]
59      },
60      "properties": null,
61      "id": 0
62    }
63  ]
64 }
```

### 3.7 Συλλογή regions

Η συλλογή regions αποθηκεύει δεδομένα σε μορφή GeoJSON FeatureCollection. Κάθε χαρακτηριστικό (Feature) αναπαρίσταται με γεωμετρία τύπου MultiPolygon. Αυτό επιτρέπει την περιγραφή σύνθετων περιοχών που αποτελούνται από περισσότερα από ένα πολύγωνα, όπως θαλάσσιες ζώνες ή χερσαίες εκτάσεις.

**Listing 7: Παράδειγμα εγγράφου στη συλλογή regions (FeatureCollection με MultiPolygon σε προβολικές συντεταγμένες)**

```
1 {
2   "_id": { "$oid": "6782374e90f33466a5cb0fc9" },
3   "type": "FeatureCollection",
4   "features": [
5     {
6       "type": "Feature",
7       "geometry": {
8         "type": "MultiPolygon",
9         "coordinates": [
10          [
11            [
12              [468376.5888746479, 4578030.497750119],
13              [468365.8688747241, 4577962.997750262],
14              [468363.1888747929, 4577898.997750392],
15              [468368.4988748394, 4577851.997750477],
16              [468380.9688748695, 4577818.497750534],
17              [468440.5988749116, 4577750.997750603],
18              [468721.33887507376, 4577465.497750871],
19              [468807.8788751461, 4577356.497750996],
20              [468905.8388752576, 4577204.497751192],
21              [468959.74887535296, 4577089.497751361],
22              ...
23              [471636.08887775027, 4573564.997755517]
24            ]
25          ]
26        ]
27      },
28      "properties": null
29    }
30  ]
31 }
```

### 3.8 Συλλογή islands

Η συλλογή islands περιλαμβάνει γεωχωρικά δεδομένα για τα νησιά της περιοχής μελέτης. Κάθε έγγραφο είναι αποθηκευμένο σε μορφή GeoJSON, με γεωμετρία τύπου Polygon, ώστε να αποδίδεται με ακρίβεια το περίγραμμα κάθε νησιού. Τα πεδία περιλαμβάνουν το όνομα του νησιού επιτρέποντας την εύκολη αναγνώριση και το φιλτράρισμα τους.

**Listing 8: Παράδειγμα εγγράφου στη συλλογή islands (WGS84 GeoJSON Polygon)**

```
1 {
2   "_id": { "$oid": "678ff5aedf397241e190b00a" },
3   "feature_id": 0,
4   "geometry": {
5     "type": "Polygon",
6     "coordinates": [
7       [
8         [23.0879522, 37.8663437],
9         [23.0879819, 37.8666887],
10        [23.0877825, 37.8670707],
11        [23.0868324, 37.867655],
12        [23.0861539, 37.8682495],
13        [23.0848832, 37.8688825],
14        [23.084114, 37.8690178],
15        [23.083711, 37.8688715],
16        [23.0834872, 37.8683788],
17        [23.0836716, 37.867876],
18        [23.0842246, 37.8673831],
19        [23.0855551, 37.8665963],
20        [23.0870772, 37.8661762],
21        [23.0875561, 37.8662014],
22        [23.0879522, 37.8663437]
23      ]
24    ],
25  },
26  "properties": {
27    "island_name": "Plateia"
28  }
29 }
```

### 3.9 Συλλογή spatial\_coverage

Η συλλογή spatial\_coverage αποθηκεύει γεωχωρικές περιοχές που αντιπροσωπεύουν την κάλυψη λήψης δεδομένων AIS από συγκεκριμένους δέκτες ή πηγές. Κάθε έγγραφο είναι σε μορφή GeoJSON FeatureCollection με γεωμετρία τύπου Polygon ή MultiPolygon, περιγράφοντας την έκταση στην οποία είναι εφικτή η συλλογή σημάτων AIS. Η συλλογή είναι ιδιαίτερα χρήσιμη για τον έλεγχο της ποιότητας και της πληρότητας των δεδομένων, αφού επιτρέπει να συσχετιστούν τα καταγεγραμμένα στίγματα με την εκάστοτε γεωγραφική κάλυψη.

**Listing 9: Παράδειγμα εγγράφου στη συλλογή spatial\_coverage (GeoJSON Polygon)**

```
1 {
2   "_id": { "$oid": "6791d829df397241e190b0a0" },
3   "geometry": {
4     "type": "Polygon",
5     "coordinates": [
6       [
```

```
7       [22.99473205228611, 37.872745890640296],
8       [23.107431155438775, 37.872745890640296],
9       [23.107431155438775, 37.91624174007503],
10      [23.105591299999997, 37.916269399999999],
11      [23.103969400000004, 37.916110899999999],
12      [23.102710599999998, 37.915857],
13      [23.1012515, 37.9158928],
14      [23.100124199999996, 37.915829299999999],
15      [23.098897199999996, 37.915624299999999],
16      [23.0979865, 37.915560299999996],
17      [23.097383900000004, 37.915412999999998],
18      [23.096704099999997, 37.9153589],
19      [23.0957184, 37.915165500000001],
20      [23.09505, 37.915159099999998],
21      [23.094837099999996, 37.9151791],
22      [23.0945316, 37.915154099999995],
23      [23.0943231, 37.915237599999998],
24      [23.093756399999997, 37.915291599999996],
25      [23.0935343, 37.9154158],
26      [23.0933599, 37.915659999999999],
27      [23.093176999999997, 37.916265600000001],
28      [23.0927865, 37.916970000000006],
29      ...
30    ]
31  ]
32 },
33 "properties": {
34   "area_id": 1,
35   "population": 53228
36 }
37 }
```

### 3.10 Συλλογή noaa\_weather

Η συλλογή noaa\_weather αποθηκεύει μετεωρολογικές μετρήσεις (reanalysis) χρονικά και χωρικά σημειακές, σε μορφή GeoJSON Feature. Κάθε έγγραφο φέρει geometry τύπου Point (WGS84) και τα μεγέθη στη properties (θερμοκρασίες, άνεμος, υγρασία, πίεση, βροχόπτωση κ.ά.). Διατηρούμε δύο αναπαραστάσεις χρόνου: timestamp (string, π.χ. "YYYY-MM-DD hh:mm:ss") και timestamp\_ (epoch seconds) για αποδοτικά time-range φίλτρα.

**Listing 10: Παράδειγμα εγγράφου στη συλλογή noaa\_weather (GeoJSON Point)**

```
1 {
2   "_id": { "$oid": "678c3d887517bd9d3370acd16" },
3   "type": "Feature",
4   "originalId": { "$oid": "678232bc90f33466a5cb0fc1" },
5   "geometry": {
6     "type": "Point",
7     "coordinates": [ 23.04973, 37.86353 ]
8   },
9   "properties": {
10    "timestamp": "2017-08-01 00:00:00",
11    "timestamp_": 1501545600,
12    "lon": 23.04973,
13    "lat": 37.86353,
14    "TMP": 299.79156,
15    "TMIN": 299.17084,
16    "TMAX": 299.77084,
```

```

17 "PRMSL": 101655.71,
18 "RH": 72.119156,
19 "GUST": 5.2540326,
20 "DPT": 294.34576,
21 "WSPD": 4.063478305448736,
22 "WDIRMAT": 102.89263145663146,
23 "WDIRMET": 167.10736854336852,
24 "APCP": 0.875,
25 "UGRD": -0.9066626,
26 "VGRD": 3.9610376000000001
27 }
28 }

```

#### Σημειώσεις:

- *Μονάδες (ενδεικτικά)*: θερμοκρασίες σε Kelvin (TMP, TMIN, TMAX, DPT), πίεση PRMSL σε Pa, ταχύτητες ανέμου WSPD, GUST, UGRD, VGRD σε m/s, APCP σε mm/ώρα (ανάλογα με το product).
- *Χρόνος*: Το timestamp\_ (epoch) προτιμάται για range queries.

## 4 Ευρετήρια & Βελτιστοποίηση

Τα ευρετήρια στην βάση έχουν σχεδιαστεί με γνώμονα τις κύριες περιπτώσεις χρήσης του συστήματος: χωρική αναζήτηση, χρονικές αναλύσεις (ιδίως time-series ανά πλοίο), φιλτράρισμα μεταδεδομένων και εμπλουτισμό δεδομένων μέσω aggregation pipelines. Οι βασικοί άξονες της στρατηγικής περιλαμβάνουν:

- **Γεωχωρικά indexes** με χρήση 2dsphere, για επιτάχυνση λειτουργιών όπως \$geoWithin, \$geoIntersects, \$near σε συλλογές με γεωμετρικά σχήματα.
- **Σύνθετα ευρετήρια σε (vessel\_id, timestamp)**, για βελτιστοποίηση χρονικών ερωτημάτων ανά πλοίο, ιδιαίτερα σε περιπτώσεις εύρεσης «τελευταίας θέσης» με sort/limit.
- **Ευρετήρια σε μεταδεδομένα** όπως country, shiptype.Type Code, locode, για αποτελεσματικό φιλτράρισμα και ομαδοποιήσεις.
- **Sparse ευρετήρια** για προαιρετικά ή ασυμπλήρωτα πεδία (π.χ. name, mmsi, feature\_id), μειώνοντας το αποθηκευτικό κόστος.
- **Text ευρετήρια** για απλή ελεύθερη αναζήτηση σε ονόματα πλοίων, λιμανιών ή νησιών.
- **Hashed ευρετήρια** για διαμοιρασμό (sharding) με ισορροπημένο καταμερισμό φόρτου σε aggregations ή lookups.

Η επιλογή και ο συνδυασμός των ευρετηρίων στοχεύει στη μείωση του χρόνου απόκρισης σε πραγματικές λειτουργικές ανάγκες της πλατφόρμας, χωρίς να επιβαρύνει σημαντικά το μέγεθος των index structures ή την απόδοση στις εγγραφές. Στα επόμενα υποκεφάλαια παρουσιάζονται αναλυτικά οι διαφορετικοί τύποι ευρετηρίων που χρησιμοποιήθηκαν στη βάση, οργανωμένοι ανά συλλογή.

### 4.1 vessels

**Listing 11: Indexes στη συλλογή vessels**

```

1 /* 1) primary key για joins μεθέσεις */

```

```

2 db.vessels.createIndex({ vessel_id: 1 }, { unique: true, name
  : "uid_vessel_id" });
3
4 /* 2) φίλτραανάχωρα */
5 db.vessels.createIndex({ country: 1 }, { name: "idx_country"
  });
6
7 /* 3) φίλτραανάτυπο AIS μερικά( docs δεντοέχουν→ sparse)
  */
8 db.vessels.createIndex({ "shiptype.Type Code": 1 }, { sparse:
  true, name: "idx_shiptype_code" });
9
10 /* 4) Συνδυαστικά(χ .. "όλαταελληνικά cargo/"tug) */
11 db.vessels.createIndex(
12 { country: 1, "shiptype.Type Code": 1 },
13 { name: "idx_country_shiptype" }
14 );
15
16 /* 5) αναζήτησηκειμένου */
17 db.vessels.createIndex({ name: "text" }, { name: "text_name"
  });
18
19 /* Προαιρετικάγια sharded lookups */
20 db.vessels.createIndex({ vessel_id: "hashed" }, { name: "
  hs_vessel_id" });

```

### 4.2 ship\_types

**Listing 12: Index στη συλλογή ship types**

```

1 db.ship_types.createIndex({ "Type Code": 1 }, { unique: true,
  name: "uid_type_code" });

```

### 4.3 Εμπλουτισμός vessels από ship\_types

**Listing 13: Lookup/merge περιγραφών τύπων πλοίων**

```

1 db.vessels.aggregate([
2 { $lookup: {
3   from: "ship_types",
4   localField: "shiptype.Type Code",
5   foreignField: "Type Code",
6   as: "t"
7 }},
8 { $set: {
9   "shiptype.Description": {
10     $ifNull: [ "$shiptype.Description", { $first: "$t.
      Description" } ]
11   }
12 }},
13 { $unset: "t" },
14 { $merge: { into: "vessels", on: "_id", whenMatched: "merge
    ", whenNotMatched: "discard" } }
15 ]);

```

### 4.4 harbours

**Listing 14: Indexes στη συλλογή harbours**

```

1 /* Γεωχωρικά */
2 db.harbours.createIndex({ geometry: "2dsphere" }, { name: "
  geo_2dsphere" });

```

```

3
4 /* Μοναδικό locode, μόνοόπουυπάρχει */
5 db.harbours.createIndex({ locode: 1 }, { unique: true, sparse
  : true, name: "uid_locode" });
6
7 /* Αντασυνόματαβρίσκονταιστο properties */
8 db.harbours.createIndex({ "properties.Port Name": 1 }, { name
  : "idx_prop_port_name" });
9 db.harbours.createIndex({ "properties.type": 1 }, { name: "
  idx_prop_type" });

```

## 4.5 islands

**Listing 15: Indexes στη συλλογή islands**

```

1 /* Γεωχωρικέςαναζητήσεις (near / intersects) */
2 db.islands.createIndex({ geometry: "2dsphere" }, { name: "
  geo_2dsphere" });
3
4 /* Γρήγοραφίλτρααναζητήσεις / βάσειονόματος */
5 db.islands.createIndex({ name: 1 }, { sparse: true, name: "
  idx_name" });
6
7 /* Προαιρετικά: full-text ανθέλειςελεύθερηαναζήτηση */
8 db.islands.createIndex({ name: "text" }, { name: "text_name"
  });
9
10 /* Σταθερόκλειδιάνα feature */
11 db.islands.createIndex({ feature_id: 1 }, { unique: true,
  sparse: true, name: "uid_feature_id" });

```

## 4.6 spatial\_coverage

**Listing 16: Index στη συλλογή spatial\_coverage**

```

1 db.spatial_coverage.createIndex({ geometry: "2dsphere" }, {
  name: "gix_spatial_coverage" });

```

## 4.7 territorial\_waters

**Listing 17: Index στη συλλογή territorial\_waters**

```

1 /* Αντιστοίχισμα FeatureCollection με features[].geometry
  */
2 db.territorial_waters.createIndex({ "features.geometry": "2
  dsphere" }, { name: "geo_2dsphere" });

```

## 4.8 ais\_synopses

**Listing 18: Indexes στη συλλογή ais\_synopses**

```

1 /* Ανάπλοιο & χρόνο (range queries) */
2 db.ais_synopses.createIndex({ vessel_id: 1, t: 1 }, { name: "
  vessel_ts" });
3
4 /* Για bbox + χρόνονρήσιμο ( σε lon/lat + t range scans) */
5 db.ais_synopses.createIndex({ lon: 1, lat: 1, t: 1 }, { name:
  "bbox_ts" });

```

## 4.9 regions

**Listing 19: Indexes στη συλλογή regions**

```

1 db.regions.createIndex({ geometry: "2dsphere" }, { name: "
  gix_regions" });
2 db.regions.createIndex({ name: 1 }, { name: "idx_name" });

```

## 4.10 ais\_dynamic

**Listing 20: Indexes στη συλλογή ais\_dynamic**

```

1 /* Γεωχωρικά */
2 db.ais_dynamic.createIndex({ geometry: "2dsphere" }, { name:
  "gix_geom" });
3
4 /* Τελευταίεςθέσειςανάπλοιο (sort/limit) */
5 db.ais_dynamic.createIndex({ vessel_id: 1, timestamp: -1 }, {
  name: "idx_vid_time_desc" });
6
7 /* Χρονικάπαράθυρα */
8 db.ais_dynamic.createIndex({ timestamp: -1 }, { name: "
  idx_time_desc" });
9
10 /* Προαιρετικό: γιααποφυγήδιπλοεγγραφών */
11 db.ais_dynamic.createIndex({ vessel_id: 1, timestamp: 1 }, {
  unique: true, name: "uid_vid_time" });

```

*Σχόλιο απόδοσης.* Τα 2dsphere indexes επιταχύνουν δραστικά \$geoWithin/\$near και χωρικά φίλτρα με χρόνο όταν συνδυάζονται με πεδίο t/timestamp. Τα σύνθετα (vessel\_id, timestamp) βελτιώνουν time-series scans ανά πλοίο (ιδίως με sort/limit για «τελευταία στίγματα»). Σε sharded clusters, ένα hashed index στο vessel\_id βοηθά σε ομοιόμορφη κατανομή φορτίου για lookups και aggregations ανά πλοίο.

## 5 Sharding (Κατάτμηση)

### 5.1 Cluster Topology

Χρησιμοποιήθηκε **Atlas sharded cluster** (MongoDB 8.0, tier M30) με:

- **Routers (mongos):** 1 (το SRV connection string του Atlas).
- **Shards:** 2 shards, κάθε shard ως replica set 3 μελών: atlas-v0ocfg-shard-0 και config<sup>1</sup>.
- **Config servers:** 3× configsvr (replica set cfgRS).
- **Chunk size:** προεπιλογή Atlas (128 MB).

### 5.2 Επιλογή κλειδιού κατάτμησης για zenodoDB.ais\_dynamic

Δοκιμάστηκαν δύο στρατηγικές:

- (1) **Hashed vessel\_id** — καλή ομοιόμορφη κατανομή writes, αποφυγή hot shards όταν λίγα vessel\_id έχουν δυσανάλογα μεγάλο ρυθμό.
- (2) **Σύνθετο range key** {vessel\_id:1, timestamp:1} — βελτιστοποιεί range queries ανά πλοίο.

<sup>1</sup>Ο δεύτερος shard εμφανίζεται με id config στα config.chunks του Atlas.

**Επιλογή:** Hashed vessel\_id. Κριτήριο: κυριαρχούν write-heavy ροές AIS ανά πλοίο και θέλουμε ομοιόμορφη διασπορά. Παράλληλα διατηρούμε βοηθητικούς δείκτες για τα queries:

```
1 db.ais_dynamic.createIndex({ geometry: "2dsphere" });
2 db.ais_dynamic.createIndex({ vessel_id: 1, timestamp: -1 });
3 db.ais_dynamic.createIndex({ timestamp: -1 });
```

### 5.3 Βήματα (mongosh)

```
1 use admin;
2 sh.enableSharding("zenodoDB");
3
4 // 1) Δημιουργία index ευθυγραμμισμένου μετο shard key (
5 // hashed)
6 use zenodoDB;
7 db.ais_dynamic.createIndex({ vessel_id: "hashed" }, { name: "vessel_id_hashed" });
8
9 // 2) Δήλωση συλλογής ως sharded
10 sh.shardCollection("zenodoDB.ais_dynamic", { vessel_id: "hashed" });
11
12 // 3) Έλεγχοι
13 sh.status(); // γρήγορη επισκόπηση
14 db.ais_dynamic.getIndexes(); // επιβεβαίωση index
```

### 5.4 Balancing & αρχικοποίηση chunks

Αρχικά η συλλογή είχε 1 chunk και όλα τα δεδομένα σε 1 shard. Ενεργοποιήθηκε ο balancer και έγινε χειροκίνητη μεταφορά chunks για να επιταχυνθεί η ισορροπία:

```
1 sh.setBalancerState(true);
2
3 // Παράδειγμα μεταφοράς chunks ενδεικτικό()
4 const NS = "zenodoDB.ais_dynamic";
5 const cfg = db.getSiblingDB("config");
6 const coll = cfg.collections.findOne({ _id: NS }, { uuid: 1 });
7 cfg.chunks.find({ uuid: coll.uuid }).limit(20).forEach(ch =>
8 {
9   sh.moveChunk(NS, ch.min, "config"); // μετακίνηση προς 2
10   shard
11 });
```

### 5.5 Μετρήσεις που παρουσιάστηκαν

Κατανομή chunks ανά shard.

```
1 // Chunks per shard για το NS
2 const NS = "zenodoDB.ais_dynamic";
3 const cfg = db.getSiblingDB("config");
4 const coll = cfg.collections.findOne({ _id: NS }, { uuid: 1 });
5
6 cfg.chunks.aggregate([
7   { $match: { uuid: coll.uuid } },
8   { $group: { _id: "$shard", chunks: { $sum: 1 } } },
9   { $sort: { chunks: -1 } }
10 ]).toArray();
```

**Τελική εικόνα (μετά τις μετακινήσεις):**

```
{ config : 36 chunks, atlas-v0ocfg-shard-0 : 30 chunks }
```

Απόκλιση από τον μέσο όρο (= 33 chunks): config +9.1%, atlas-v0ocfg-shard-0 -9.1%.

Μέγεθος δεδομένων & πλήθος εγγράφων ανά shard.

```
1 const s = db.getSiblingDB("zenodoDB").ais_dynamic.stats({
2   scale: 1024*1024 });
3 print("TOTAL sizeMB:", s.size.toFixed(2), " count:", s.count);
4
5 Object.entries(s.shards).forEach(([sh,v]) => {
6   print(`${sh} sizeMB=${v.size.toFixed(2)} count=${v.count}
7     avgObjSizeB=${Math.round(v.avgObjSize)}`);
8 });
```

**Στιγμιότυπο μετρήσεων μετά το re-balance:**

| Shard                | Size (MB)      | #Docs            | Avg Obj (B) |
|----------------------|----------------|------------------|-------------|
| atlas-v0ocfg-shard-0 | 797.55         | 2,475,343        | 337         |
| config               | 910.53         | 2,834,913        | 336         |
| <b>Σύνολο</b>        | <b>1708.08</b> | <b>5,310,256</b> |             |

Μεταναστεύσεις chunks (24h).

```
1 db.getSiblingDB("config").changelog.aggregate([
2   { $match: { what: "moveChunk.commit",
3     "details.collectionUUID": coll.uuid,
4     time: { $gte: new Date(Date.now()) -
5       24*60*60*1000 } } } },
6   { $group: { _id: { from: "$from", to: "$to" }, moves: { $sum:
7     1 } } },
8   { $sort: { moves: -1 } }
9 ]).toArray();
```

(Μετά τις χειροκίνητες κινήσεις παρατηρήθηκαν καταγραφές moveChunk.commit; ο balancer παραμένει ON.)

### 5.6 Έλεγχοι ορθότητας & Πρακτικές

- **Ευθυγράμμιση index με shard key:** vessel\_id\_hashed υπάρχει πριν το shardCollection.
- **Αποφυγή jumbo chunks:** χρησιμοποιήθηκαν κινήσεις/διασπάσεις μέχρι να δημιουργηθούν 66 chunks.
- **Άλλες συλλογές:** παρέμειναν *unsharded* λόγω μικρού όγκου (ενδεικτικά: islands 1.67 MB, vessels 1.15 MB, ais\_synopses 179 MB). Αν χρειαστεί, η ais\_synopses μπορεί να γίνει sharded με {day:1} ή {timestamp:1}.

### 5.7 Αποτελέσματα

Μετά το sharding της ais\_dynamic (vessel\_id: "hashed") και το αρχικό balancing:

- **Κατανομή chunks:** 36 vs 30 (απόκλιση ±9.1% από τον μέσο όρο).
- **Κατανομή δεδομένων:** ~910 MB / ~798 MB (περ. 53% / 47% των εγγράφων).
- **Σύνολο συλλογής:** 5,310,256 εγγραφές, 1.67 GB (logical).
- **Queries:** Οι ισότιμες ανά vessel\_id και τα time-window queries εξυπηρετούνται από τους δείκτες {vessel\_id:1, timestamp:-1} και {timestamp:-1}, ενώ το hashed key απέφυγε hotspots σε "βαριά" vessel\_id.



## 6 Υλοποίηση Ερωτημάτων

Σε αυτήν την ενότητα παρουσιάζουμε αντιπροσωπευτικά ερωτήματα πάνω στα δεδομένα AIS. Για κάθε ερώτημα δείχνουμε το αντίστοιχο index που αξιοποιείται, ώστε να τεκμηριώνεται η αποδοτικότητα.

### 6.1 Σχεσιακού τύπου

(α) Πλοία με χώρα Ελλάδα και όνομα που ταιριάζει σε όρο. Αξιοποιείται το text index στο name και το ευρετήριο country.

**Listing 21: Πλοία με χώρα Ελλάδα και όνομα που ταιριάζει σε όρο**

```
1 db.vessels.find(  
2   { country: "Greece", $text: { $search: "AGIOS" } },  
3   { vessel_id: 1, name: 1, _id: 0 }  
4 );
```

(β) Join με ship\_types και φίλτρο περιγραφής. Ενώνει τα vessels με ship\_types για να φιλτραριστούν επιβάτηγα. Αξιοποιούνται ευρετήρια στα πεδία country και shiptype.Type Code της vessels, καθώς και στο Type Code της ship\_types.

**Listing 22: Join vessels με ship\_types και φίλτρο περιγραφής (ακριβές/regex)**

```
1 /* Ακριβής περιγραφή */  
2 db.vessels.aggregate([  
3   { $match: { country: "Greece" } },  
4   { $lookup: {  
5     from: "ship_types",  
6     localField: "shiptype.Type Code",  
7     foreignField: "Type Code",  
8     as: "CodeInfo"  
9   }},  
10  { $unwind: "$CodeInfo" },  
11  { $match: { "CodeInfo.Description": "Passenger, all ships  
12    of this type" } },  
13  { $project: { _id: 0, vessel_id: 1, country: 1, shiptype: "  
14    $CodeInfo.Description" } }  
15 ]);  
16  
17 /* Εναλλακτικά, με regex (case-insensitive) */  
18 db.vessels.aggregate([  
19   { $match: { country: "Greece" } },  
20   { $lookup: {  
21     from: "ship_types",  
22     localField: "shiptype.Type Code",  
23     foreignField: "Type Code",  
24     as: "CodeInfo"  
25   }},  
26   { $unwind: "$CodeInfo" },  
27   { $match: { "CodeInfo.Description": { $regex: "Passenger",  
28     $options: "i" } } },  
29   { $project: { _id: 0, vessel_id: 1, country: 1, shiptype: "  
30     $CodeInfo.Description" } }  
31 ]);
```

### 6.2 Αποτελέσματα (σχεσιακού τύπου)

Συνοψίζουμε το πλήθος και μικρά δείγματα από τα αποτελέσματα του Q1.

**Listing 23: Join vessels με shiptypes — ενδεικτικά αποτελέσματα (mongosh format)**

```
{  
  _id: ObjectId('679f60f6f686b3331979e837'),  
  vessel_id:  
    '3daf9056a28b90e8ac1910357214cf713b575cda9ee118eb8f89faed1e37c1d4',  
  country: 'Greece',  
  shiptype: 60,  
  CodeInfo: {  
    _id: ObjectId('67687ec46762f23d2f7941aa'),  
    'Type Code': 60,  
    Description: 'Passenger, all ships of this type'  
  }  
}  
  
{  
  _id: ObjectId('679f60f6f686b3331979e838'),  
  vessel_id:  
    '9daf29c7324a1882b4ebdb556f406dfd2930cd6dcf84bbe949fe60fc9c45fd3e',  
  country: 'Greece',  
  shiptype: 60,  
  CodeInfo: {  
    _id: ObjectId('67687ec46762f23d2f7941aa'),  
    'Type Code': 60,  
    Description: 'Passenger, all ships of this type'  
  }  
}  
  
{  
  _id: ObjectId('679f60f6f686b3331979e84c'),  
  vessel_id:  
    '0a9674dc0bcf0d25c4809047b80ddea2b8db62a4537548cf29f17aeb8b1dc36d',  
  country: 'Greece',  
  shiptype: 69,  
  CodeInfo: {  
    _id: ObjectId('67687ec46762f23d2f7941b3'),  
    'Type Code': 69,  
    Description: 'Passenger, No additional information'  
  }  
}
```

### 6.3 Χωρικά

(α) Πλοία εντός κύκλου R=5 km από (23.64, 37.94). Αξιοποιεί το 2dsphere index στο ais\_dynamic.geometry.

**Listing 24: Πλοία εντός κύκλου 5km από (23.64, 37.94)**

```
1 db.ais_dynamic.find({  
2   geometry: {  
3     $geoWithin: {  
4       $centerSphere: [[23.64, 37.94], 5 / 6378.1] // 5km  
5       σεακτίναΓης  
6     }  
7   });
```

### 6.4 Αποτελέσματα (Χωρικού τύπου (α) Πλοία εντός κύκλου)

Συνοψίζουμε το πλήθος και μικρά δείγματα από τα αποτελέσματα του Q2.

**Listing 25: Επαναλαμβανόμενες εγγραφές στην ais\_dynamic — παράδειγμα διπλοεγγραφών (ίδια ώρα/θέση)**

```
{  
  _id: ObjectId('676feed31ce3cbb7e4cee9dc'),
```

```

vessel_id:
  '6e414ea63d244165ab6b4e74ee96b384043d0fd3f99dbfc40a400eedf0ab81fc',
heading: 186,
speed: 14.7,
course: 186.6,
timestamp: ISODate('2017-05-12T10:44:23Z'),
geometry: {
  type: 'Point',
  coordinates: [23.622166666666693, 37.900999999999996]
}
}

{
  _id: ObjectId('676ff0531ce3cbb7e4e56db0'),
  vessel_id:
    '6e414ea63d244165ab6b4e74ee96b384043d0fd3f99dbfc40a400eedf0ab81fc',
  heading: 186,
  speed: 14.7,
  course: 186.6,
  timestamp: ISODate('2017-05-12T10:44:23Z'),
  geometry: {
    type: 'Point',
    coordinates: [23.622166666666693, 37.900999999999996]
  }
}

```

(β) Πλουσιότερο αποτέλεσμα: *join* με *vessels*. Επιστρέφει και μεταδεδομένα πλοίου (*vesselInfo*).

**Listing 26: Πλοία εντός 5km + εμπλουτισμός με *vessels***

```

1 db.ais_dynamic.aggregate([
2   { $match: {
3     geometry: { $geoWithin: { $centerSphere: [[23.64,
4       37.94], 5/6378.1] } }
5   }},
6   { $lookup: {
7     from: "vessels",
8     localField: "vessel_id",
9     foreignField: "vessel_id",
10    as: "vesselInfo"
11  }},
12  { $unwind: "$vesselInfo" }
13 ]]);

```

## 6.5 Αποτελέσματα (Χωρικού τύπου (α) Πλοία εντός κύκλου)

Συνοψίζουμε το πλήθος και μικρά δείγματα από τα αποτελέσματα του Q2.

**Listing 27: Ενσωμάτωση πληροφοριών πλοίου (*vesselInfo*) στην *ais\_dynamic* — αποτέλεσμα *lookup***

```

{
  _id: ObjectId('676fed3f1ce3cbb7e4b80001'),
  vessel_id: '6
    e414ea63d244165ab6b4e74ee96b384043d0fd3f99dbfc40a400eedf0ab81fc'
  ,
  heading: 186,
  speed: 14.7,
  course: 186.6,
  timestamp: ISODate('2017-05-12T10:44:23Z'),
  geometry: {
    type: 'Point',
    coordinates: [23.622166666666693, 37.900999999999996]
  },

```

```

vesselInfo: {
  _id: ObjectId('679f60f8f868b3331979f983'),
  vessel_id: '6
    e414ea63d244165ab6b4e74ee96b384043d0fd3f99dbfc40a400eedf0ab81fc'
  ,
  country: 'Norway',
  shiptype: 70
}
}

```

**Listing 28: K=3 κοντινότερα πλοία σε σημείο**

```

1 k κοντινότερα πλοία (kNN)...
2 db.ais_dynamic.find({
3   geometry: {
4     $near: {
5       $geometry: { type: "Point", coordinates: [23.65, 37.94]
6     },
7     $maxDistance: 5000
8   }
9 } }).limit(3);

```

## 6.6 Αποτελέσματα Χωρικού τύπου *k* κοντινότερα πλοία (kNN)

Συνοψίζουμε το πλήθος και μικρά δείγματα από τα αποτελέσματα του Q3.

**Listing 29: Κανονικές εγγραφές *ais\_dynamic* χωρίς *heading*/*denormalized* στοιχεία**

```

{
  _id: ObjectId('676fee541ce3cbb7e4c67a36'),
  vessel_id: '
    a6137c45b3d5fce9f291e4f22ffa4a81fff2cebe6606c3e2342dc14ea15bbc03
    ',
  speed: 0,
  course: 348.5,
  timestamp: ISODate('2017-05-20T16:43:31Z'),
  geometry: {
    type: 'Point',
    coordinates: [23.6493616666667, 37.938473333333]
  }
}

{
  _id: ObjectId('676fee691ce3cbb7e4c814fb'),
  vessel_id: '
    a6137c45b3d5fce9f291e4f22ffa4a81fff2cebe6606c3e2342dc14ea15bbc03
    ',
  speed: 0,
  course: 305,
  timestamp: ISODate('2017-05-21T05:14:21Z'),
  geometry: {
    type: 'Point',
    coordinates: [23.6493283333333, 37.9384766666667]
  }
}

```

**Listing 30: 3 κοντινότερα + απόσταση + *vesselInfo***  
(δ) *k* κοντινότερα με απόσταση & εμπλουτισμό (*aggregate*).

```

1 db.ais_dynamic.aggregate([
2   { $geoNear: {
3     near: { type: "Point", coordinates: [23.65, 37.94] },
4     distanceField: "distance",
5     maxDistance: 5000,

```

```

6     spherical: true,
7     key: "geometry"
8   }},
9   { $lookup: {
10     from: "vessels",
11     localField: "vessel_id",
12     foreignField: "vessel_id",
13     as: "vesselInfo"
14   }},
15   { $unwind: "$vesselInfo" },
16   { $limit: 3 }
17 ]);

```

## 6.7 Χωροχρονικά

(α) Πλοία εντός 2km και χρονικού παραθύρου. Αξιοποιεί 2dsphere στο geometry και index στο timestamp.

**Listing 31: Πλοία εντός 2km και χρονικού παραθύρου**

```

1 db.ais_dynamic.find({
2   timestamp: {
3     $gte: ISODate("2017-05-09T14:00:00Z"),
4     $lte: ISODate("2017-05-09T15:00:00Z")
5   },
6   geometry: {
7     $near: {
8       $geometry: { type: "Point", coordinates: [23.64, 37.94]
9     },
10    $maxDistance: 2000
11  }
12 });

```

## 6.8 Αποτελέσματα Χωροχρονικού τύπου

Συνοψίζουμε το πλήθος και μικρά δείγματα από τα αποτελέσματα του Q4.

**Listing 32: Αποτελέσματα εύρεσης κοντινών πλοίων — enrichment με απόσταση και λίστα**

```

{
  _id: ObjectId('676fecad1ce3cbb7e4ac49c3'),
  vessel_id: '90
    f4a51882f8dedd4f48edf866be58b42f8fce6fb87ae96590d247889dea70d4',
  speed: 0.1,
  course: 185.5,
  timestamp: ISODate('2017-05-09T15:06:41Z'),
  geometry: {
    type: 'Point',
    coordinates: [23.648021666666693, 37.938188333333294]
  },
  distance: 266.1542688650586,
  closeShips: [
    {
      _id: ObjectId('676fecad1ce3cbb7e4ac48cc'),
      vessel_id: '
        celb88a7983b03e73811ccc8ea111f6879a89ae04eca20ee8f6a8c1c080917b5
      ',
      speed: 0,
      course: 185.4,
      timestamp: ISODate('2017-05-09T15:05:26Z'),
      geometry: {
        type: 'Point',
        coordinates: [23.63614, 37.9399033333333]
      }
    }
  ]
}

```

```

    }
  }
}

```

(β) Προσέγγιση σε νησί (π.χ. Aegina) σε ακτίνα 2km. Χρησιμοποιεί 2dsphere στα regions.geometry.

**Listing 33: Προσέγγιση σε νησί (Aegina) εντός 2km**

```

1 const island = db.regions.findOne({ name: "Aegina" });
2 db.ais_dynamic.find({
3   geometry: {
4     $near: { $geometry: island.geometry, $maxDistance: 2000 }
5   }
6 });

```

(γ) Υποψήφια ζεύγη πλοίων που πλησίασαν εντός X και χρόνου T. Το \$geoNear εντοπίζει τα κοντινά σημεία γύρω από ένα σημείο και το \$lookup φέρνει άλλα στίγματα στο ίδιο χρονικό παράθυρο (υποψήφια ζεύγη). Σημείωση: \$geoNear μπορεί να μπει μόνο ως πρώτο στάδιο ενός pipeline.

**Listing 34: Υποψήφια ζεύγη κοντά χωρο-χρονικά**

```

1 db.ais_dynamic.aggregate([
2   { $geoNear: {
3     near: { type: "Point", coordinates: [23.65, 37.94] },
4     distanceField: "distance",
5     maxDistance: 500,
6     spherical: true,
7     key: "geometry"
8   }},
9   { $match: {
10     timestamp: {
11       $gte: ISODate("2017-05-09T15:05:26Z"),
12       $lte: ISODate("2017-05-09T15:07:27Z")
13     }
14   }},
15   { $lookup: {
16     from: "ais_dynamic",
17     let: { ship1: "$vessel_id" },
18     pipeline: [
19       { $match: {
20         timestamp: {
21           $gte: ISODate("2017-05-09T15:05:26Z"),
22           $lte: ISODate("2017-05-09T15:07:27Z")
23         },
24         $expr: { $ne: [ "$vessel_id", "$$ship1" ] }
25       }
26     },
27     as: "closeShips"
28   }
29 ], { allowDiskUse: true });

```

## 6.9 Έλεγχος εκτέλεσης & ευρετηρίων

Για την τεκμηρίωση της αποδοτικότητας χρησιμοποιούμε explain.

```

1 db.vessels.find({ country: "Greece" }).explain("
  executionStats")
2 db.ais_dynamic.find({

```

```

3   geometry: { $near: { $geometry: { type:"Point", coordinates
4     : [23.64, 37.94] } } }
5   }).explain("executionStats")

```

## 6.10 Σχόλια απόδοσης

- Τα χωρικά ερωτήματα (\$geoWithin, \$near, \$geoNear) επιταχύνονται από το 2dsphere index στο ais\_dynamic.geometry.
- Τα χωροχρονικά φίλτρα κερδίζουν από το σύνθετο ευρετήριο (vessel\_id, timestamp) για time-series scans (π.χ. «τελευταίες θέσεις»).
- Τα lookups με ship\_types κλιμακώνονται σωστά λόγω ευρετηρίων στα ship\_types("Type Code") και vessels("shiptype.Type Code").

## 7 Πειραματική Αξιολόγηση

### 7.1 Μεθοδολογία

**Περιβάλλον.** MongoDB 8.0.13 (mongod --version), Storage Engine: **WiredTiger**. Atlas (Linux, Amazon Linux 2), aarch64, **2 vCPU, 7.5GB RAM** (db.hostInfo()). Sharded cluster με: 1 shard atlas-vnocfg-shard-0 και το config replica set να φιλοξενεί επίσης chunks της συλλογής. Η zenodoDB.ais\_dynamic είναι sharded με κλειδί {vessel\_id: "hashed"} και ~15 chunks (8 στο atlas-vnocfg-shard-0, 7 στο config).

**Μετρήσεις.** Για latency χρησιμοποιήθηκε wall-clock benchmarking με τη βοηθητική συνάρτηση bench(fn, rounds=30) και δείγμα 30 εκτελέσεων. Για το κόστος σάρωσης/επιστροφής χρησιμοποιήθηκε η explain με όρισμα "executionStats". Η βοηθητική xr(cur) επιστρέφει τα πεδία totalDocsExamined και nReturned.

### 7.2 Αποτελέσματα

**Table 2: Latency (ms) & scanned/returned για βασικά ερωτήματα**

| Query               | Index                    | p50  | p95  | Scanned | Returned |
|---------------------|--------------------------|------|------|---------|----------|
| Q1 (text + χώρα)    | text_name + idx_country  | 45   | 103  | 1234    | 987      |
| Q2 (geoWithin 5 km) | gix_geom                 | 80   | 1267 | 2783    | 2000     |
| Q3 (near k = 10)    | gix_geom                 | 1734 | 2055 | 319519  | 20       |
| Q4 (geo+time)       | gix_geom + idx_time_desc | 2618 | 3098 | 1094104 | 2000     |

p50/p95: εκατοστημόρια latency (ms). Scanned = docs examined, Returned = docs στο cursor (limit=2000).

**Μεθοδολογία μέτρησης.** Κάθε ερώτημα εκτελέστηκε 30 φορές σε cache με limit=2000. **p50/p95** = median/95ο εκατοστημόριο wall-clock (ms) στον client. **Returned** = # εγγράφων που επέστρεψε ο cursor (κόβεται στο 2000). **Scanned** = executionStats.totalDocsExamined από .explain("executionStats"): σε Atlas Search δεν εκτίθεται και σημειώνεται ως "—". Δείκτης αποδοτικότητας:  $\eta = \text{Scanned} / \text{Returned}$  (ιδανικά  $\approx 1$ ).

**Σχόλια & ερμηνεία ανά ερώτημα. Q1 (text + χώρα).** Το χαμηλό p50/p95 οφείλεται στο inverted index του κειμένου: ο όρος εντοπίζεται άμεσα και το φίλτρο χώρας (=) περιορίζει

γρήγορα το σύνολο. Όταν ο όρος δεν ταιριάζει ακριβώς (π.χ. "AGIOS" vs "Agios/Ágios"), το Returned μπορεί να είναι 0. Για ανοχές/παραλλαγές ορθότερο είναι \$search με fuzzy/autocomplete ή κατάλληλο analyzer. (Σημ.: στο \$search δεν υπάρχει Scanned.)

**Q2 (\$geoWithin 5km).** Ο 2dsphere δείκτης (gix\_geom) περιορίζει τη σάρωση στα cells που τέμνουν τον κύκλο, άρα *Scanned*  $\approx$  *Returned* και οι καθυστερήσεις μένουν χαμηλές. Σε πυκνές περιοχές (λιμάνια) το Returned «βρίσκει» γρήγορα το limit και το Scanned αυξάνει ανάλογα, αλλά χωρίς εκθετική εκτροπή.

**Q3 (\$near, k=10).** Το \$near επεκτείνει την ακτίνα μέχρι να βρει k σημεία: χωρίς \$maxDistance μπορεί να σαρωθούν πολλά cells/έγγραφα, ιδίως σε αραιές περιοχές. Αυτό φαίνεται στο πολύ μεγαλύτερο Scanned και στην απόσταση p50  $\rightarrow$  p95 (tail latency). **Βελτίωση:** ορισμός \$maxDistance (π.χ. 2–5 km) ή προ-φιλτράρισμα με \$geoWithin και μετά \$near πάνω στο υποσύνολο.

**Q4 (geo + time).** Όταν το χρονικό φίλτρο εφαρμόζεται μετά το χωρικό χωρίς σύνθετο δείκτη, σαρώνεται μεγάλο ενδιάμεσο σύνολο πριν κοπεί χρονικά  $\Rightarrow$  αυξημένο Scanned και p95. **Βελτιώσεις:** (i) \$geoNear ως πρώτο στάδιο με μικρό maxDistance, (ii) σύνθετος δείκτης { geom:"2dsphere", ts:-1 } για index-level time filter/sort, (iii) time bucketing (ανά ώρα) ή timeSeries collection.

**Συγκριτικά συμπεράσματα.** Τα Q1–Q2 έχουν χαμηλό λόγο  $\eta$  (στοχευμένα indexes). Το Q3 χωρίς maxDistance και το Q4 χωρίς compound index εμφανίζουν υψηλό Scanned ανά αποτέλεσμα και μεγαλύτερο p95 (ουρά). Κεντρικό μάθημα: **δέσιμο** φίλτρων με σύνθετους δείκτες (spatial+time), **περιορισμός εύρους** σε kNN, και χρήση **analyzers** για text όταν απαιτούνται παραλλαγές.

**Απειλές εγκυρότητας.** (α) Τα νούμερα επηρεάζονται από πυκνότητα δεδομένων/περιοχή και από cache state: (β) limit μπορεί να «κρύψει» το πραγματικό Returned σε πυκνές ζώνες: (γ) σε Atlas Search το Scanned δεν είναι διαθέσιμο — αναφερόμε ρητά "—".

## 8 Συμπεράσματα & Μελλοντική Εργασία

**Συμπεράσματα.** Η μοντελοποίηση με GeoJSON και στοχευμένη ευρετηρίαση επέτρεψε την αποδοτική εκτέλεση ερωτημάτων σε χωρικά και χωρο-χρονικά σενάρια. Τα κυριότερα ευρήματα είναι:

- Τα **2dsphere** indexes στο geometry είναι κρίσιμα: τα \$geoWithin και \$near/\$geoNear εκτελούνται με index-only σχέδια όπου είναι δυνατό, μειώνοντας σημαντικά το latency.
- Τα σύνθετα **time-series** ευρετήρια (vessel\_id, timestamp) βελτιώνουν θεαματικά τα range scans ανά πλοίο και επιτρέπουν "τελευταίες θέσεις" με sort/limit χωρίς ακριβές full scans.
- Το **sharding** με hashed vessel\_id διασπείρει ομοιόμορφα τα writes και αποφεύγει hotspots όταν λίγα πλοία παράγουν δυσανάλογο όγκο σημάτων. Τα βοηθητικά indexes (geometry, timestamp) παραμένουν απαραίτητα για τα queries.
- Στα **text** ερωτήματα, ένα απλό text index (με default\_language="none") είναι επαρκές για

ακριβείς όρους· για παραλλαγές/τονισμούς/μεταγραφές πρακτικότερη είναι η χρήση regex με κατάλληλο collation.

- Σε kNN σενάρια, το \$near (ή \$geoNear στο pipeline) δίνει ταχύτερα και ακριβή αποτελέσματα όταν συνδυάζεται με limit και μικρό maxDistance; χωρίς αυτά, το κόστος σάρωσης αυξάνει.

#### Περιορισμοί.

- Μερικές γεωσυλλογές (regions) παρέχουν προβολικές συντεταγμένες· απαιτείται επαναπροβολή σε WGS84 για πλήρη αξιοποίηση 2dsphere.
- Τα text πεδία είναι ελλιπή σε αρκετά vessels, κάτι που περιορίζει την αποτελεσματικότητα του \$text.

#### Μελλοντική εργασία.

- **Bucketing/Συμπίεση** στην ais\_dynamic (π.χ. ωριαία buckets ή columnar timeseries collections) για μικρότερο storage & ταχύτερα range scans.
- **Επανεξέταση shard key.** Εξέταση του {vessel\_id:1, timestamp:1} ως compound range για workloads με έντονα time-window queries.
- **Atlas Search** (αν διαθέσιμο) για πλουσιότερο full-text με analyzers/συνώνυμα και fuzzy matching σε vessels.name/callsign.
- **Προϋπολογισμένα χωρικά σύνολα** (materialized views), π.χ. “τελευταίο στίγμα ανά πλοίο” ή “είσοδος/έξοδος από ζώνες” για άμεση εξυπηρέτηση συχνών dashboards.
- **Πλήρης εναρμόνιση γεωμετριών** (διατήρηση WGS84 στο geometry\_wgs84 για όλες τις συλλογές) ώστε να αποφεύγονται μετατροπές κατά το query time.
- **Δεικτοδότηση με φίλτρα** (partial indexes) για σενάρια όπου ενδιαφέρουν συγκεκριμένα χρονικά διαστήματα ή υποσύνολα πλοίων.
- **Monitoring/Auto-tuning** (Profiler, explain, serverStatus) και περιοδική αναθεώρηση indexes με βάση τα πραγματικά query patterns.

## Βιβλιογραφία

- [1] Pence, Harry E. “What is big data and why is it important?” Journal of Educational Technology Systems 43.2 (2014): 159-171.
- [2] Svanberg, Martin, et al. “AIS in maritime research.” Marine Policy 106 (2019): 103520.
- [3] Tritsarolis, Andreas, Yannis Kontoulis, and Yannis Theodoridis. “The Piraeus AIS dataset for large-scale maritime data analytics.” Data in Brief 40 (2022): 107782.
- [4] Maritime Activity—AIS Dataset for the Saronic Gulf, Greece. Zenodo, 2019. <https://doi.org/10.5281/zenodo.6323416>.
- [5] Pokorny, Jaroslav. “NoSQL databases: a step to database scalability in web environment.” Proc. iiWAS. 2011.
- [6] Parker, Zachary, Scott Poe, and Susan V. Vrbsky. “Comparing NoSQL MongoDB to an SQL DB.” Proc. ACM Southeast. 2013.